

adoption of QUIC.

In QUIC the initial handshake happens directly with the TLS 1.3 handshake.

Application layer

Telnet vs SSH

Telnet (Teletype network) and SSH (Secure Shell) are protocols used to connect and access the remote system. We can use them to connect to the remote system to perform actions like executing commands, file changes, file copy and move, and so on.

Telnet doesn't provide authentication and the data is transferred in plain text. SSH is similar to Telnet but is more secure. It consists of three distinct layers.

- **Authentication layer:** Authenticates the user.
- **Connection layer:** Manages the communication between the two endpoints after the authentication succeeds, such as opening and closing of communication channels, allowing multiple channels for multiple sessions, and so on.
- **Transport layer:** After the channel is established for communication, the data encryption, decryption and integrity are taken care of in this layer.

SSH flow: The client contacts the server to initiate a connection. The server responds by sending the client a public cryptography key. The server negotiates parameters and opens a secure channel for the client. The server and client perform asymmetrically to exchange symmetric keys. The user authentication occurs using password and SSH keys. After authentication succeeds, the user, through their client, logs into the server.

SSH uses hashing to make sure the data doesn't lose its integrity. It uses algorithms for data encryption such as IDEA, Blowfish, DES, 3DES, etc.

FTP vs FTPS/SFTP

FTP or File Transfer Protocol is used for file transfer over the network. It passes the data along with user name and password in plain text, which is pretty risky. FTPS was implemented to overcome this challenge, providing secure connections using SSL/TLS in the transport layer. SFTP or SSH File Transfer Protocol was also implemented to provide security, and uses SSH to transfer the files. Both FTPS and SFTP provide similar functionality using different protocols.

DNS vs DNSSEC

Domain Name System (DNS) was originally designed as an open protocol. DNS servers are organised in a hierarchy. At the top level of the hierarchy, root servers store the complete database of the internet domain names and their

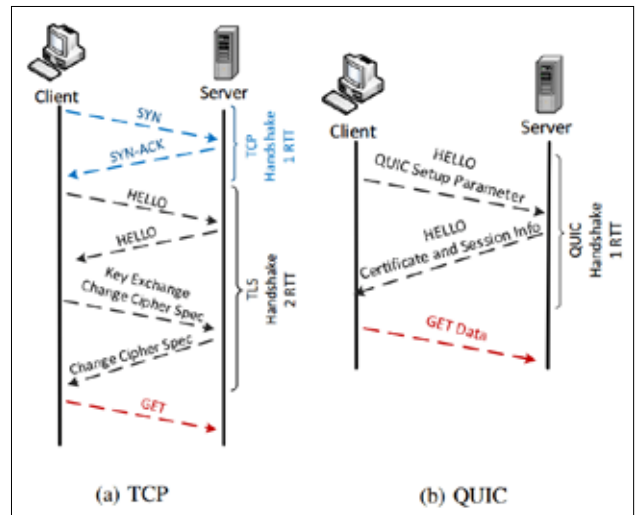


Figure 5: Handshaking of HTTP2-over-TCP and QUIC protocols (Credits: <https://www.researchgate.net/profile/Anees-Al-Najjar/publication/346652270/figure/fig14/AS:965459260227618@1607194833532/Handshaking-of-HTTP2-over-TCP-and-QUIC-protocols.ppm>)

corresponding IP addresses.

Different types of DNS servers exist, of which two are common.

Authoritative servers: These servers are definitive for particular domains and provide information about only those domains. An authoritative-only name server returns answers to queries about the domain names that have been specifically configured.

Caching or recursive servers: These servers use recursion to resolve a given name, from the DNS root through to the authoritative name servers of the queried domain.

DNS does not check for credentials before accepting an answer, and attackers exploit this basic vulnerability. An attacker can cause DNS poisoning by delegating a false name to the domain server and providing a false address for the server.

To prevent this from happening, the Domain Name System Security Extensions (DNSSEC) protocol was developed. DNSSEC protects against such attacks by providing a validation path for records. With DNSSEC, validation is done using a key and a signature.

To properly validate the path, DNSSEC must be implemented at each domain level.

The higher organisation level signs the key of the lower domain level. For example, with the domain `www.example.com`, the root level signs the `.com` key and the `.com` level signs the `example.com` key.

DNSSEC follows the chain of trust from the lowest-level domain to the top-level domain, validating the keys at each level along the way. It was developed to strengthen DNS through digital certificates, public keys and private